

Object-Oriented Design Principles

How Do You Design?

- What principles guide you when you create a design?
- What considerations are important?
- When have you done enough design and can begin implementation?

What Have You Heard About Design?

- First principles of OOP

- Find the nouns → objects/state
- Find the verbs → behaviors; methods/functions
- Encapsulation, inheritance
- Programming

- Object Interactions

- Static → collaboration diagrams
- Dynamic → sequence diagrams

What is the Next Level of Design?

- How do you decide on the class collaborations?
 - Simple: Follow principles of design
- But there are problems here!
 - What if the principles conflict with each other?
 - A good class decomposition may have too many connections between classes
 - Strong encapsulation may lead to performance penalties
 - Does each new design start from first principles?
 - Abstraction has been our friend
 - It can help again
 - How can we reliably follow the principles?
 - Learn from previous designers
 - Stand on each others' shoulders, not each others' toes.

Key Design Concepts

- **General**

- Cohesion
- Coupling
- Information hiding

- **OO Specific**

- Class vs. Interface Inheritance
- Inheritance / composition / delegation

Key OO Design Principles

- Object Oriented programming's goal is to make it possible to write code that is maintainable, easy to understand, and reusable.
- Used incorrectly, you can write OO code that is as difficult to maintain, understand and reuse as you can with procedural code.
- Following these OO Design Principles will help you write better OO code.

Principles of Successful Design

- Increase cohesion where possible
 - Separation of concerns
 - Focused experts
- Decrease coupling where possible
 - Simplify interfaces
 - Reduce connections in number and volume
- Employ and support reuse
 - Reuse existing designs and code where possible
 - Increase reusability where possible
 - Design for portability

Principles of Successful OO Design

1. Encapsulate what varies.
2. Program to an interface, not an implementation
3. Favor Composition over Inheritance
4. Strive for loosely coupled designs between objects that interact
5. Classes should be open for extension, but closed for modification
6. Depend upon abstractions, not upon concrete classes
7. Principle of least knowledge – talk only to your immediate friends
8. Hollywood Principle – Don't call us, we'll call you
9. A class should have only one reason to change

SOLID Principle

- S- (SRP) Single Responsibility
- O- (OCP) Open-Closed Principle
- L- (LSP) Liskov Substitution Principle
- I- (ISP) Interface Segregation
- D-(DIP) (Dependency Inversion Principle)

OOP Principle

- DRY- Don't Repeat Yourself
- KISS- Keep It Simple, Stupid
- YAGNI- You Aren't Gonna Need It
- CRUD- Create, Read, Update and Delete

1. Encapsulate what varies

- Identify the aspects of your application that vary and separate them from what stays the same.
- Take the parts that vary and encapsulate them, so that later you can alter or extend the parts that vary without affecting the parts that don't.
- Reduces the chances of breaking some other part of your app when you make a change.

2. Program to an interface, not an implementation

- Client app is decoupled from knowing the details of the concrete implementation.
- If you see “As New” (or = New), you are creating a concrete implementation.

3. Favor Composition over Inheritance

- “Has A” vs. “Is A”
 - Generally done by keeping an instance of another class within your class, and using its properties and methods.
- You can only inherit from one class, you can compose a class of as many other classes as you want.
- Lets you add functionality to a class without changing the class.

4. Strive for loosely coupled designs between objects that interact

- Lets you write code for a service without knowing who is going to use the service, or how it is going to use it.
- The event model are examples of this design principle.

5. Classes should be open for extension, but closed for modification

- The idea is to get the class right, and lock it down. Make future changes through extending the class with new classes.
- Applying this principle where it is not needed is unnecessary work, and can lead to complex, hard to understand code.

6. Depend upon abstractions, not upon concrete classes

- Ideal:

- No variable should hold a reference to a concrete class
- No class should derive from a concrete class
- No method should override an implemented method of its base class (if it is something that can be changed, separate it into its own class).

7. Principle of least knowledge – talk only to your immediate friends

- The more classes your classes interact with, the more fragile, and hard to understand it becomes.
- Have your classes only talk to:
 - Their own methods
 - Objects passed in as parameters
 - Objects the object creates
 - Components of the object

8. Hollywood Principle – Don't call us, we'll call you

- When a high level object instantiates a low level object, the low level object just does what it is asked to do. It does not ask the high level object to do anything.

9. A class should have only one reason to change

- Plan your classes based on what is likely to change. If there is more than one thing that may change, split the class until there is only one thing likely to change in each class.

Goals in OO Development

- Cohesion: a measure of how closely a class or module supports a single purpose or responsibility (happy, low stress objects). You want this to be high.
- Coupling: a measure of the number of classes a class has references to. You want this to be low.
- Design Patterns are strategies for accomplishing these goals.